PROGRAM GUIDE

The path from first code to research

Computer science + computer engineering for games

Read this guide to choose a starting point, follow prerequisites, plan study hours and assess real engineering work.

200 lessons / 600 quiz questions / 10 volumes

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

What this program is

This is a guided self-study curriculum for software engineers who want to understand and build game engines, work close to gaming hardware, or progress toward hardware architecture and digital design. It contains 50 modules, 200 lessons and 600 lesson-quiz questions across ten volumes. Every lesson supplies an outcome, an original teaching note, a worked example, a practical assignment with a pass condition, assigned reading and three questions. Explanatory answers are in a separate key and expandable in the browser edition.

A lesson is a **multi-session study unit**, not a 20-minute substitute for a university subject. The teaching notes introduce the concepts; the assigned chapters, lectures, problem sets, implementations and reviews provide the depth. Reading this package without doing that work will not produce undergraduate or graduate competence. External textbooks and courses are linked rather than redistributed, so the package is not an entirely offline replacement for those materials.

The breadth was cross-checked against ACM/IEEE-CS/AAAI CS2023 and the ACM/IEEE computer-engineering curriculum report CE2016. This is not an accredited degree, a university-approved equivalency, professional licensure, security certification or authorization to sign off production silicon. An institution or employer may assess the work independently under its own rules. [R01, R02]

The default route assumes no programming experience and basic reading and arithmetic. The learner's age, equipment, mathematics background and weekly availability were not supplied. No age-dependent assumptions are made. A beginner can start with the supplied first-day program; a more experienced learner can use the placement rules below. Electronics begins in simulation, with physical extensions only after a qualified safety review.

Learning outcomes and professional direction

By the undergraduate software checkpoint, the learner should be able to design, implement, test, profile and distribute a small original game and engine; explain the relevant mathematics; and reason about memory, algorithms, concurrency, networks and resource ownership. By the undergraduate hardware checkpoint, the learner should be able to specify and verify a digital subsystem, connect it to software, interpret synthesis/timing evidence and distinguish modeled claims from physical measurements.

Graduate work adds derivation, formal or numerical reasoning, strong baselines, research reproduction, controlled experiments, uncertainty analysis and a defended thesis-style contribution. The full route covers both engine and hardware breadth; the thesis should pursue **one primary specialization**, not attempt a whole commercial engine and a new console simultaneously.

Direction	Areas to emphasize after the shared foundation	Strong portfolio evidence
Engine / systems engineer	Volumes 3-6; Modules 37-39, 42, 44-45	Resource-safe engine, task graph, profiler, reproducible release.
Rendering / GPU engineer	Mathematics; Modules 21-23, 37-38, 41-43, 45	Renderer, derived sampling method, correct GPU primitive and equal-quality comparison.
Platform / performance engineer	Volumes 4, 7-8; Modules 42, 46, 48	Latency trace, memory investigation, driver model and reliable cross-platform boundary.
CPU / GPU / accelerator designer	Volumes 2, 4, 7-8; Modules 41-42, 46-48	Verified RTL, architectural model, timing evidence and complete host interface.
Tools / compiler engineer	Modules 11-15, 20, 25, 41, 45, 50	Asset pipeline, editor commands, typed DSL and semantics-preserving optimization.

These are emphasis choices, not permission to bypass a listed prerequisite. Engine-focused learners do not need to wait until the thesis to start building relevant portfolio work. Hiring standards vary; no completion point here guarantees a job.

Program map and planning hours

All hour figures are **author planning estimates**, not measured completion times or transferable academic credits. They include assigned study, problem solving, labs, review and the corresponding checkpoint project. Do not add all project hours again on top of the table. Prior preparation, mentorship, debugging and project scope can move actual time substantially. Volume 10 includes the 400-800 hour thesis planning range.

Volume	Modules	Main level and focus	Estimated hours
1	01-05	Beginner programming, debugging, testing and first game	220-340
2	06-10	Discrete math, linear algebra, calculus, probability and signals	500-800

Volume	Modules	Main level and focus	Estimated hours
3	11-15	C++, ownership, algorithms, data structures and languages	400-650
4	16-20	Architecture, OS, concurrency, networking and engineering	380-600
5	21-25	CPU/GPU graphics, materials, engine architecture and tools	420-650
6	26-30	Physics, animation, audio, AI, multiplayer and release	420-650
7	31-35	Circuits, logic, RTL, processors and FPGA subsystems	450-720
8	36-40	Boards, GPU architecture, performance and co-design	350-560
9	41-45	Graduate numerics, GPU algorithms, rendering and compilers	550-850
10	46-50	Graduate hardware, verification, research thesis and defense	810-1,380
Full dual-breadth route	01-50	All 200 lessons and Gates A-J	4,500-7,200

At 30 focused hours per week for 46 weeks per year, the full estimated range is about **3.3-5.2 years**. At 15 hours per week for 46 weeks, it is about **6.5-10.4 years**. These are arithmetic illustrations, not a promised schedule. A specialist route, substantial prior knowledge or separately completed university subjects can change the workload, but skipping evidence requirements does not create equivalent mastery.

Treat this as four broad stages: foundations (Volumes 1-4), undergraduate engine practice (5-6), undergraduate computer engineering (7-8), then graduate specialization and thesis (9-10). A conventional institution may distribute the subjects differently. The sequence here optimizes for one learner building an integrated portfolio, not for a particular university's catalog.

Prerequisites and sequencing

The default sequence is numerical: 01.1 through 50.4. The digits before the decimal identify the module; the final digit identifies its lesson. Each module's opening page states prerequisites. Complete the practical checkpoint at the end of each volume before using its results as a foundation for the next stage.

Mathematics can run in parallel with early programming after Module 02, but the programming prerequisites of individual math labs must still be met. Do not begin advanced graphics without vectors, transforms and the relevant calculus/probability. Do not begin concurrency without memory and lifetime reasoning. Do not begin physical board work before the circuits and safety review. Graduate modules require the undergraduate evidence, not only recognition of terminology.

When a prerequisite is weak, return to its lesson and solve a new problem. Use a prerequisite repair log: missing skill, smallest review source, transfer exercise, result and date. A tutor or peer reviewer can help identify the actual gap. Starting at the beginning is not required for someone who can demonstrate the listed work independently.

Entry diagnostic: locate the starting point

Complete these without searching or using AI. This is placement information, not a pass/fail judgment. The answers are in the answer-key appendix under “Entry diagnostic,” separate from the 600 lesson questions.

1. Evaluate 3 + 4 x 2 using ordinary arithmetic precedence.
2. Convert 0.25 seconds to milliseconds.
3. Solve 3x + 2 = 14.
4. Evaluate 2^8.
5. Add vectors (-2,5) and (3,-1).
6. Find the length of vector (3,4).
7. Start at position 4, move right 3, then left 1. What is the final position?
8. For integer values: x=2; y=x+3; x=9. What is y?
9. What values does Python range(1,4) supply?
10. In the list [10,20,30], what is the value at zero-based index 1?
11. A while loop repeats while remaining>0 but never changes remaining or exits. What can go wrong?
12. Evaluate True and False.
13. A door opens at level 5 or above. Give three particularly useful boundary-test levels.
14. A program cannot find a relative filename. What environment fact should be checked first?
15. A program is fast on one input but produces a wrong result on another supported input. Is that an acceptable optimization?

With fewer than five correct answers on questions 1-6, schedule an algebra/trigonometry bridge using OpenStax Precalculus 2e while beginning Volume 1. Start with arithmetic, equations, functions, exponents, coordinate geometry and trigonometry; then retest. Questions 5-6 can also be learned in Module 07. This result must not block the first programming session. [R40]

With fewer than five correct answers on questions 7-12, begin at Lesson 01.1. Strong results can justify moving faster, but not skipping the labs automatically. Questions 13-15 probe engineering habits that should develop throughout the course.

Prior-learning challenge process

A self-reported skill list is not enough to skip a volume. Attempt its lesson quizzes without notes, complete the checkpoint artifact or a comparable existing artifact, and explain a new variation to a reviewer. Require the same practical standards as the ordinary route. Record what was assessed and what was not.

For the math volume, the challenge must include a proof, a transform derivation, calculus/ODE reasoning and a probability or statistical interpretation; a multiple-choice vocabulary test is insufficient. For systems, include a lifetime bug, a concurrency argument, a network failure and a measured performance explanation. For digital design, include RTL semantics, interface verification, setup/hold reasoning and an explicit reset policy. Graduate credit within this self-study plan requires research-level evidence, not only professional seniority.

The study loop and assessment rules

For each lesson: retrieve prerequisite ideas from memory, read the teaching note, predict the worked example, complete the assigned source study, perform the lab, answer the quiz, then check the key. Save evidence rather than only checking a completion box. Required source readings are identified by R-codes linked in the resource directory. A chapter's explanation does not replace its exercises.

A suggested study block is 60-90 minutes with a break: roughly 10 minutes recall, 20-30 minutes reading/derivation, 25-40 minutes implementation or problems, then a brief journal entry. This is a planning suggestion, not a medical or cognitive prescription. Use a pace and breaks appropriate to the learner.

Each short quiz has three questions scored 0-2 each. Use 0 for absent/incorrect, 1 for partial or unexplained and 2 for correct with reasoning. A formative pass is **5/6 or better**, completed assigned study and the practical pass condition. The questions are deliberately short; they are not psychometrically validated exams or sufficient evidence of degree-level mastery by themselves.

For each missed concept, write the misconception and solve a changed example. Recheck after 24-72 hours when practical. At the end of a module, demonstrate a new input, requirement or failure without following the original walkthrough. An evaluator may accept a different technically sound answer than the key when the assumptions and reasoning are explicit.

Required depth beyond the short quizzes

At minimum, each programming/systems module should produce the stated project, **6-10 additional exercises**, and **five new edge or failure cases** beyond copied examples. Every mathematics module should include **12-20 written problems**, at least two derivations or proofs where appropriate, and a computational check. Select problems covering each named reading topic; use official solutions only after an independent attempt.

For graphics and simulation modules, preserve reference images or trajectories, numerical tests, a performance/quality comparison and one failure case. For digital-design modules, include a written interface, directed boundary tests, reproducible randomized tests, assertions, and tool reports appropriate to that module. For each graduate technical module, add **6-10 substantial analytical problems**, a critical reading of at least one original paper or equivalent graduate chapter, and a reproducible experiment with a baseline.

These are minimum planning requirements, not a reason to stop when understanding is incomplete. A mentor may substitute a comparably demanding assignment. Problems should come from the referenced course or book where possible and be logged by source, section and exercise number; this avoids pretending a one-page introduction replaces a semester's mathematical practice.

The mathematics extensions named in Volume 2 mean: R40 for algebra/trigonometry, R41 for multivariable derivatives and constrained differentials, and R42 for the later graduate optimization treatment. Complete the relevant R41 sections before gradient-based graduate work. Use R54 for deeper numerical methods. [R40-R42, R54]

For hardware/firmware direction, add a mandatory C-language bridge after Module 15: compile a small C source file as C, expose a C-compatible interface to C++, document allocation/free ownership, inspect structure layout and test error returns. C and C++ are distinct languages; do not treat compiling C code as C++ as proof of compatibility. Review the chosen compiler's C documentation and the target RTOS examples before Module 36. [R11, R53]

Review rubrics and academic integrity

Use this common project rubric, adapted to the project: correctness and requirements 30 points; tests/verification 25; reasoning and design tradeoffs 15; quantitative quality/performance/error analysis 10; reproducibility 10; communication and provenance 10. Suggested pass: **80/100**, with all essential practical checks complete and no unresolved critical defect inside the claimed operating envelope. A high visual-quality score cannot compensate for an unsafe memory interface or a false correctness claim.

For graduate artifacts: contribution/question clarity 10; mathematical or technical rigor 20; implementation correctness 20; evaluation/baselines/uncertainty 25; reproducibility 15; writing and defense 10. Suggested pass: 80/100 plus independent reproduction of the central claim. Publication acceptance and positive results are not required. Clear scope, honest failure analysis and defensible evidence are required.

Use AI as a tutor, reviewer or brainstorming aid, not an unexamined answer source. Predict before asking for code, identify generated sections, check provenance, test outputs and explain

the implementation. Never upload private employer code, proprietary SDK material, credentials or other people's personal data without authorization. Supplied starter code is a learning reference; running it does not complete the learner's own project requirements.

Gate A: a complete first game

After Volume 1. Build a terminal grid or resource game with a clear initial state, commands, valid/invalid input behavior, a success/failure condition and restart. Keep the simulation separate from display. Use the included game as a reference, then implement meaningful changes rather than submitting it unchanged.

Deliver the source, README, a rule specification, at least 12 learner-written tests, a saved input sequence and its expected final state, and a short journal explaining two bugs. Include tests for boundaries, blocked movement, invalid commands, restart and repeated rendering without state changes. Replaying the same inputs in the pinned environment must produce the expected result. A second person should run a complete loop from the README.

The oral check asks the learner to change one rule, predict affected tests and repair the implementation. Passing does not require graphics. Do not add networking, a database or a large engine merely to make the project look advanced.

Gate B: a game-mathematics notebook

After Volume 2. Produce a versioned notebook or scripts plus written derivations. Required artifacts are a logic/invariant proof, vector and matrix operations, a transform-chain example, a one-dimensional dynamics model, a sampling/aliasing demonstration and a small statistically analyzed experiment.

Tests must include a known transform, a zero-vector edge case, an analytic motion comparison, a timestep refinement study and a probability calculation checked by simulation. Show units and numerical tolerances. Do not normalize a zero vector silently or treat a random experiment as proof. Complete and log the required source problem sets.

The oral check supplies a changed coordinate convention, a new timestep and a misleading average. The learner must explain what changes and what remains invariant. A screenshot of a plot without derivation or reproducible data is insufficient.

Gate C: native simulation and a tiny language

After Volume 3. Implement the core simulation in C++ with explicit ownership and a build system, then add a small expression language or bytecode interpreter. It may evaluate movement or material expressions; it must have a specified grammar, type/error behavior and resource limits.

Deliver a clean build, tests, memory/lifetime checks available on the toolchain, a complexity analysis and a comparison of two appropriate data structures. Include empty input, malformed tokens, invalid bytecode or expressions, lifetime boundaries, container invalidation and numerical limits. Add the C-language bridge for the hardware direction.

The oral check includes one pointer/lifetime defect, a data-structure choice and a new grammar case. The learner should explain the entire compile/link/run path. A library may supply support utilities, but not replace the central learning implementation without an explicit comparison exercise.

Gate D: a defensible systems application

After Volume 4. Build a local job-based application that reads validated work, performs bounded processing and stores results. A small local client/server or task-processing tool is enough; public deployment is unnecessary.

Deliver a requirements sheet, architectural decision record, thread/resource ownership map, protocol/schema, threat model, automated build/test script, dependency inventory and recovery report. Test malformed messages, duplicate requests, cancellation, partial writes, worker failure and shutdown under load. Run relevant memory/concurrency diagnostics where supported and record their limitations.

The oral check asks why the design is race-free under its stated model, how retries avoid duplicated durable effects and what evidence identifies the measured bottleneck. A functioning happy path alone does not pass.

Gate E: a small 3D engine

After Volume 5. Build an engine demonstration containing a CPU reference renderer, one supported GPU backend where available, a fixed-step simulation, explicit entity/resource ownership, an asset cooker, an inspector and versioned save/load. A software-only execution path is acceptable while GPU access is unavailable, but untested GPU claims must remain excluded.

Deliver a fixed scene, source assets with provenance, build/cook commands, validation logs, headless tests, screenshot or image-difference evidence, frame-time data and a peak-memory report. The budget is chosen for the declared test machine; 60 fps is an example target, not a universal pass requirement. Test near-plane crossing, resize/minimize, stale handles, malformed assets and resource shutdown.

A second person must build from a clean checkout or extracted source tree. The learner must explain one CPU/GPU dependency and one quality/performance tradeoff. Using an existing engine for comparison is encouraged; using it in place of the central engine implementation changes the learning objective and needs a separate implementation project.

Gate F: undergraduate software capstone

After Volume 6. Release a small original game built on the engine. Include one finished gameplay loop; collision/physics; animation or articulated motion; bounded audio; an inspectable AI behavior; settings; save/load; and a tested package. The multiplayer requirement can be a separate local two-client demonstration with delay, loss and replay tests rather than a public online service.

Evidence includes a compatibility matrix, remappable controls, readable interface tests, alternative presentation for a critical audio cue, pseudo-localization, a clean-folder launch and a technical postmortem. Preserve CPU/GPU or software-renderer timing boundaries, raw frame distributions and memory measurements. Test interrupted saves, invalid mods if modding is exposed, controller/device absence and exit during loading.

A reviewer must finish the loop, reproduce a bug test and observe a live change. The quality bar is a coherent, testable small system. It is not commercial art quality, a large open world or a revenue target.

Gate G: undergraduate digital-design capstone

After Volume 7. Implement one RTL subsystem: a synchronous queue with a small processor interface, a sprite compositor, a fixed-point vector block or another similarly bounded game-related function. The module-level processor exercise remains required even when the capstone centers on a different block.

Deliver the interface specification, software reference, RTL, automated testbench, directed and randomized cases, assertions, reset review and synthesis reports. Test empty/full, simultaneous operations, backpressure, overflow/rounding and reset while work is pending where applicable. Label simulation, synthesis and physical measurements separately.

No board purchase is required. A board demonstration is optional and follows the safety plan. Passing FPGA operation does not establish ASIC readiness. The oral check asks for a waveform explanation, a nonblocking-assignment trace and setup versus hold reasoning.

Gate H: undergraduate hardware/software co-design capstone

After Volume 8. Choose one measured engine workload. Compare a competent software baseline with at least two architecture alternatives, implementing one alternative as RTL, a supported simulation model or a bounded analytical prototype with clear limitations.

Required evidence includes workload distributions, numerical-quality limits, host/device ownership, setup and transfer costs, resource/timing estimates, failure handling and a sensitivity analysis. Evaluate small and large workloads and one adverse case. A negative conclusion is acceptable: proving that a software optimization is preferable is valid co-design work.

The review must identify the maximum application benefit, the dominant bottleneck, which results are modeled and which are measured, and what changes under a tighter memory or latency constraint. Do not compare an optimized accelerator against an intentionally weak software baseline.

Gate I: graduate reproduction and extension

After Volume 9. Select a bounded published or graduate-level method in rendering, simulation, GPU algorithms or compilation. Reconstruct the method, reproduce one central result and add a clearly identified extension, new setting or rigorous failure analysis. The extension need not be globally novel; its relation to prior work must be honest.

Deliver a claim-evidence map, derivation or semantic argument, implementation, independent reference or baseline, raw data, analysis scripts, one ablation and one parameter sensitivity study. Use equal-quality comparisons when quality changes with speed. Report uncertainty at the appropriate trial unit and avoid training/tuning on the final evaluation set without disclosure.

A reviewer should reproduce the main table or result. Unresolved discrepancies must be explained and narrow the claim. This checkpoint is the prerequisite for the thesis, not a replacement for it.

Gate J: thesis, defense and handoff

After Volume 10. Complete one primary thesis-scale project. Plan roughly 400-800 focused hours within the volume's estimate, but adjust the scope through review rather than treating the hours as a requirement to spend. A 30-60 page thesis-style report plus technical appendices is a useful planning target; evidence quality matters more than length.

Deliver a literature map, reviewed proposal, running baseline, risk register, versioned decisions, final method, tests, evaluation, reproducible artifact and maintainer handoff. Hold a 30-45 minute defense with a qualified reviewer or peer panel, including a derivation, a failed case and a live implementation question. Independently reproduce the central result and complete revisions.

One thesis is enough. The learner need not invent both a new engine and a new GPU. Completion records should say self-study curriculum and describe the actual artifacts, not claim an accredited bachelor's or master's degree.

Thesis project menu: choose one primary question

Area	Example question	Essential evidence
Rendering	Can a sampling/reconstruction change reduce equal-error time on a defined moving-scene set?	Estimator derivation, reference images, motion failures, total cost and uncertainty.
Engine scheduling	Can a bounded scheduling policy reduce p99 frame time during streaming bursts without exceeding memory limits?	Task traces, competent baselines, burst workloads, memory peaks and ablations.

Area	Example question	Essential evidence
Physics	How do timestep, iteration count and compliance affect error and cost in a small deformable simulation?	Equations, gradient/constraint checks, reference solution and sensitivity study.
GPU algorithms	Does a compaction or wavefront strategy improve a specified irregular workload after queue overhead?	Work/span reasoning, termination and bounds tests, skewed workloads and end-to-end timings.
Accelerator design	Does a verified fixed-point block meet a game-quality tolerance and improve end-to-end latency or energy?	Numerical error, RTL tests/properties, host interface, PPA assumptions and transfer cost.
Architecture / compilers	Does a cache, layout or specialization policy improve a stated game-workload family?	Model validation, semantic correctness, held-out workloads and Pareto tradeoffs.

These are question templates, not claims that the techniques are new or guaranteed to succeed. Review relevant newer research when the project begins. The 2016 XPBD and 2020 ReSTIR references are stable historical anchors for reproduction, not a survey of the latest work. [R44, R55]

Hardware, software and safety policy

Start with the computer already available. The first-day Python work and early programming do not require a dedicated GPU, FPGA, oscilloscope, cloud subscription or game-engine license. Use a code editor, a supported Python 3 interpreter and later Git. At Module 11, add a C++20-capable compiler and CMake. Record the actual versions rather than upgrading tools in the middle of a reproducibility experiment. [R04, R05, R11, R12]

At graphics, choose one supported path: Vulkan, native Direct3D 12 on a compatible Windows setup, or Metal on a compatible Apple setup. Check actual device and tutorial capabilities before installation. A moving “latest” tutorial may assume newer features than the learner's device supports. SDL3 examples must use SDL3 APIs rather than mixing SDL2 instructions. Keep the CPU renderer as a reference and fallback. [R21, R22, R24, R25]

CUDA is an optional vendor-specific route, not a requirement for learning GPU algorithms. Use compute shaders or a supported alternative where appropriate, and explicitly limit claims when a GPU is unavailable. Use official platform documentation rather than leaked console SDKs. No supplied lesson grants proprietary access or distribution rights. [R21, R24, R25, R37]

For digital design, begin with supported RTL simulation and synthesis tools such as Verilator, cocotb, Yosys and SymbiYosys; compatibility depends on versions and language features. Board selection should follow the project and toolchain, not precede them. For architecture and

physical design, use gem5 and an educational OpenROAD-compatible flow only within their documented model and library scope. [R32-R34, R38, R39, R50]

Physical experiments require a qualified mentor, exact board/peripheral documentation, compatible voltage levels and a low-voltage safety plan. Keep protective mechanisms enabled. Do not open or modify mains power supplies, CRTs, high-energy battery packs or high-voltage equipment. Do not design a first project around mains circuitry. Do not assume a low nominal voltage removes short-circuit or thermal risks. Use reviewed commercial educational boards and approved supplies. Hardware procurement and fabrication are optional extensions, not first-day requirements.

Source use, maintenance and limitations

R-codes identify the authoritative reading directory that follows. The lesson teaching notes and quizzes are original instructional synthesis; cited resources provide primary course material, official specifications, author textbooks or original research. Resource descriptions are not claims that every linked page is a complete beginner course. For example, the Python tutorial is a reference for people with some programming familiarity; CS50 Python is the recommended beginner lecture companion. [R03, R04]

Use the source sections named in each lesson, and log the actual chapter and problem numbers studied. Some course sites are historical archives, intentionally chosen for durable foundations. Software documentation is often a moving target. Record a release or access date and use matching documentation. Where an exact page moves, navigate from the organization's official documentation rather than silently replacing it with an unreviewed tutorial.

The source directory was assembled for this edition dated September 28, 2026. It is not a guarantee that every future link, tool feature or licensing term remains unchanged. Original books, papers, software and courses retain their own licenses. They are linked, not included as copied books or bundled proprietary software. All hardware and performance examples must be revalidated for the learner's chosen system.

This curriculum emphasizes essential technical depth for game engines and gaming hardware. It does not replace a university's full general-education requirements, supervised laboratories, formal academic assessment or the specialized mentorship required for production ASIC signoff. Its strength is a coherent sequence of explicit learning outcomes and evidence-producing work, not a credential claim.

Reference directory

The readings below provide depth beyond these lesson primers. Links lead to the authors, universities, standards organizations or tool maintainers. Record an exact release, edition and access date in each project. “Latest” documentation can change; archived university courses remain useful for foundations. Links require internet access.

R01 ACM / IEEE-CS / AAAI

CS2023 knowledge areas

Breadth checklist, not a claim of accreditation.

https://csed.acm.org/knowledge-areas/

R02 ACM / IEEE Computer Society

Computer Engineering Curricula 2016

Computer-engineering breadth and laboratory expectations; historical curriculum report.

https://ieeecs-media.computer.org/assets/pdf/ce2016-final-report.pdf

R03 Harvard University

CS50 Introduction to Programming with Python

Beginner lectures and practice; complete exercises rather than only watching.

https://cs50.harvard.edu/python/

R04 Python Software Foundation

Python tutorial and language documentation

Reference companion; the tutorial itself assumes some programming familiarity.

https://docs.python.org/3/tutorial/

R05 Scott Chacon and Ben Straub

Pro Git

Version control and collaboration.

https://git-scm.com/book/en/v2

R06 MIT OpenCourseWare

6.042J Mathematics for Computer Science

Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R07 MIT OpenCourseWare

18.06 Linear Algebra

Matrices, vector spaces, eigenvalues and problem sets.

https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

R08 MIT OpenCourseWare

18.01SC Single Variable Calculus

Calculus lectures, exercises and assessments.

https://ocw.mit.edu/courses/18-01sc-single-variable-calculus-fall-2010/

R09 MIT OpenCourseWare

18.05 Introduction to Probability and Statistics

Probability, inference and quantitative reasoning.

https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

R10 MIT OpenCourseWare

6.003 Signals and Systems

Sampling, convolution, transforms and systems.

https://ocw.mit.edu/courses/6-003-signals-and-systems-fall-2011/

R11 C++ Core Guidelines editors

C++ Core Guidelines

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 Kitware

CMake tutorial

Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 MIT OpenCourseWare

6.006 Introduction to Algorithms

Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 LLVM Project

Kaleidoscope and LLVM tutorials

Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 MIT OpenCourseWare

6.004 Computation Structures

Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 Remzi and Andrea Arpaci-Dusseau

Operating Systems: Three Easy Pieces

Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R17 IETF / RFC Editor

RFC 9293: Transmission Control Protocol

Transport semantics; reference reading after networking foundations.

https://www.rfc-editor.org/rfc/rfc9293.html

R18 PostgreSQL Global Development Group

PostgreSQL tutorial

Relational data, transactions and SQL; pin local server and client versions.

https://www.postgresql.org/docs/current/tutorial.html

R19 Carnegie Mellon University

15-462/662 Computer Graphics

Rendering, geometry and animation course structure and exercises.

https://15462.courses.cs.cmu.edu/fall2021/

R20 Carnegie Mellon University

15-466/666 Computer Game Programming

Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

R21 Khronos Group

Vulkan tutorial

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

R22 SDL project

SDL3 documentation

Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples.

https://wiki.libsdl.org/SDL3/FrontPage

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys

Physically Based Rendering, fourth edition

Rendering mathematics and implementation; use chapter titles in lesson assignments.

https://www.pbr-book.org/4ed/contents

R24 Microsoft

Direct3D 12 graphics

Windows graphics backend reference.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

R25 Apple

Metal developer resources

Apple-platform graphics and compute reference.

https://developer.apple.com/metal/

R26 Epic Games

Programming with C++ in Unreal Engine

Engine comparison after C++ foundations, not a beginner substitute.

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

R27 Glenn Fiedler

Fix Your Timestep!

Original practitioner explanation of fixed-step simulation; read critically and implement your own lab.

https://gafferongames.com/post/fix_your_timestep/

R28 Box2D project

Box2D documentation

Collision, constraints and simulation reference.

https://box2d.org/documentation/

R29 Microsoft

Xbox Accessibility Guidelines

Accessibility design and testing; not a legal-compliance certificate.

https://learn.microsoft.com/en-us/xbox/accessibility/guidelines

R30 MIT OpenCourseWare

6.002 Circuits and Electronics

Circuits, transistors and laboratory reasoning.

https://ocw.mit.edu/courses/6-002-circuits-and-electronics-spring-2007/

R31 Noam Nisan and Shimon Schocken

Nand to Tetris

Computer construction projects; review the authors' license terms.

https://www.nand2tetris.org/

R32 Verilator project

Verilator user guide

RTL simulation and linting; match supported SystemVerilog subset.

https://verilator.org/guide/latest/

R33 YosysHQ

Yosys documentation

Synthesis and technology mapping; stable installed release preferred over a development snapshot.

https://yosyshq.readthedocs.io/projects/yosys/en/latest/

R34 cocotb project

cocotb documentation

Python-based HDL verification; simulator and version compatibility must be checked.

https://docs.cocotb.org/en/stable/

R35 RISC-V International

RISC-V ISA specifications

Instruction-set reference; record the exact ISA version and implemented extensions.

https://docs.riscv.org/reference/isa/

R36 KiCad project

KiCad documentation

Schematic capture, PCB layout and manufacturing outputs.

https://docs.kicad.org/

R37 NVIDIA

CUDA Programming Guide

GPU execution and memory; CUDA is an optional vendor-specific lab route.

https://docs.nvidia.com/cuda/cuda-programming-guide/

R38 gem5 project

gem5 documentation

Architecture simulation; distinguish model outputs from measured hardware results.

https://www.gem5.org/documentation/

R39 OpenROAD project

OpenROAD documentation

RTL-to-physical-design study; educational results are not production signoff.

https://openroad.readthedocs.io/en/latest/

R40 OpenStax / Rice University

Precalculus, second edition

Algebra, functions and trigonometry bridge before calculus.

https://openstax.org/details/books/precalculus-2e

R41 MIT OpenCourseWare

18.02SC Multivariable Calculus

Partial derivatives, gradients, multiple integrals and constrained differentials.

https://ocw.mit.edu/courses/18-02sc-multivariable-calculus-fall-2010/

R42 Stephen Boyd and Lieven Vandenberghe

Convex Optimization

Graduate optimization text available from its authors; copyrighted, linked rather than redistributed.

https://web.stanford.edu/~boyd/cvxbook/

R43 Samuel Williams, Andrew Waterman and David Patterson

Roofline performance model, technical report

Original 2008 performance-model reference; use current measured hardware limits for experiments.

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

R44 Benedikt Bitterli and coauthors

Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Original ReSTIR research, 2020; example reproduction target, not a claim to be the latest method.

https://research.nvidia.com/publication/2020-07_spatiotemporal-reservoir-resampling-real-time-ray-tracing-dynamic-direct

R45 LLVM Project

AddressSanitizer documentation

Memory-error instrumentation, limitations and supported toolchains.

https://clang.llvm.org/docs/AddressSanitizer.html

R46 University of California, Berkeley

CS188 Introduction to Artificial Intelligence textbook

Search, games, decision processes, learning and probabilistic reasoning.

https://inst.eecs.berkeley.edu/~cs188/textbook/

R47 Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola

Dive into Deep Learning

Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions.

https://d2l.ai/

R48 MIT OpenCourseWare

6.823 Computer System Architecture

Graduate architecture foundations; historical course concepts supplemented with current specifications.

https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

R49 Christopher Batten / Cornell University

ECE5745 Complex Digital ASIC Design

Graduate ASIC methodology, circuits, EDA and design-project expectations.

https://www.csl.cornell.edu/courses/ece5745/

R50 YosysHQ

SymbiYosys documentation

Formal hardware verification, safety, liveness and cover properties.

https://yosyshq.readthedocs.io/projects/sby/en/latest/

R51 Linux kernel contributors

Driver implementer API guide

Device ownership, I/O, DMA and driver interfaces; use disposable VMs or simulated devices for kernel experiments.

https://docs.kernel.org/driver-api/index.html

R52 Khronos Group

OpenXR registry and specifications

XR coordinate spaces, frame timing and runtime interfaces; optional hardware-dependent extension.

https://registry.khronos.org/OpenXR/

R53 Zephyr Project

Zephyr documentation

Embedded kernel, drivers, board support and simulation; select and pin a supported release.

https://docs.zephyrproject.org/latest/introduction/index.html

R54 MIT OpenCourseWare

18.085 Computational Science and Engineering I

Graduate numerical methods, equilibrium systems, differential equations and variational reasoning.

https://ocw.mit.edu/courses/18-085-computational-science-and-engineering-i-fall-2008/

R55 Matthias Mueller and collaborators

Author publication archive: position-based simulation and XPBD

Read the named 2016 XPBD paper and related course notes for the simulation reproduction; papers retain their own copyrights.

https://matthias-research.github.io/pages/publications/publications.html